UrbanSound8K dataset product (P3)

Course: IDIG4120 Digital Information Processing
Dataset: UrbanSound8K
Authors: Katharina Kivle and Nikolai O. Stubø

Datasheet reviewed by both Katharina & Nikolai.

This notebook packages the dataset together with a reproducible compression pipeline: the datasheet, the schema, the quality assurance report, the pipeline with its recorded results, and the failure handling. Every claim names its source, and where no source answers, the entry says unknown, with a note of where we looked. All recorded outputs come from one execution of this notebook against a full copy of the release, described in section 3.

Sources used:

- Dataset page: https://urbansounddataset.weebly.com/urbansound8k.html (referred to below as "dataset page")
- Dataset paper: J. Salamon, C. Jacoby and J. P. Bello, "A Dataset and Taxonomy for Urban Sound Research", 22nd ACM International Conference on Multimedia, Orlando, USA, Nov. 2014 (referred to as "dataset paper"), https://www.justinsalamon.com/uploads/4/3/9/4/4394963/salamon_urbansound_acmmm14
- Publications page of the dataset site: https://urbansounddataset.weebly.com/publications.html (referred to as "publications page")
- P1 audit report: p1/IDIG4120_P1_Kivle_Stubø.ipynb (referred to as "P1 report")
- QA report: section 3 of this notebook
- p2 documentation: p2/README.md and the module docstrings in p2/src

In [1]:
```
import sys
from pathlib import Path

# find the product root: the folder holding manifest/urbansound8k_manifest.json
root = Path.cwd()
while not (root / "manifest" / "urbansound8k_manifest.json").is_file():
    if root == root.parent:
        raise FileNotFoundError("run this notebook from inside the p3 folder")
    root = root.parent
sys.path.insert(0, str(root / "src"))

import pandas as pd
import soundfile as sf
from IPython.display import display

from p3 import config as cfg, integrity, pipeline
```

```
config = cfg.load_config(root)
dataset_dir = cfg.dataset_dir(root, config)
manifest = integrity.load_manifest(cfg.manifest_path(root))

print("dataset dir: configured in config.json")
print("manifest entries:", len(manifest["files"]))
```

```
dataset dir: configured in config.json
manifest entries: 8733
```

---

1. Datasheet

1.1 Identity and purpose

Field	Value	Source
Dataset name	UrbanSound8K	dataset page
Version	unknown	dataset page; dataset paper
Owner	unknown	dataset page; dataset paper
Creators	*J. Salamon, C. Jacoby and J. P. Bello*, the dataset paper authors	dataset paper
Contents	8,732 labeled sound excerpts (at most 4 s) of urban sounds from 10 classes, in WAV format, plus a metadata CSV	dataset page
Unit of observation	one sound excerpt (a slice); file names follow [fsID]-[classID]-[occurrenceID]-[sliceID].wav	dataset page
Intended use	research on urban sound classification, evaluated with 10-fold cross validation on the predefined folds	dataset page
Use in this product	input material for the APAC compression pipeline; no classification is performed	this repository
Out-of-scope uses	safety-critical or production use; commercial reuse (see area 4); reshuffling the predefined folds when comparing with published classification results	our assessment; fold warning on the dataset page

The ten classes are *air_conditioner, car_horn, children_playing, dog_bark, drilling, engine_idling, gun_shot, jackhammer, siren and street_music* (dataset page).

Version and owner were checked against the dataset page and the dataset paper; the dataset's authors never published a version number or named an owner, so both entries stay unknown.

1.2 Collection and sampling

Question	Answer	Source
Who or what is represented	urban sound events, recorded in the field and uploaded to freesound.org	dataset page
When	published with the dataset paper (*ACM Multimedia 2014*); the recording dates of the source material are unknown	dataset paper
How	occurrences longer than 4 s were segmented into 4 s slices with a sliding window of 2 s hop, capped at 1000 slices per class, giving 8732 slices (8.75 hours); the metadata records each excerpt's start and end time inside its source recording	dataset paper, section 3; dataset page
Sampling frame	freesound.org at collection time: over 160,000 user-uploaded recordings under a Creative Commons licence, searched per class with the class name as the query	dataset paper, section 3
Selection and exclusion criteria	every search result was manually checked by listening and by inspecting the uploader's metadata, keeping only real field recordings in which the class occurs: over 3000 recordings (60 hours) became 1302 (27 hours). Classes were chosen by their frequency in New York City's 311 noise complaints (over 370,000, 2010 to publication), except children playing and gun shot, added for variety	dataset paper, section 3
Known gaps	where and when the source recordings were made is unknown	dataset page; dataset paper
Structure	excerpts pre-sorted into ten folds (fold1 to fold10) for comparable cross-validation results: all slices from the same source recording share a fold, with slice counts per class balanced across folds	dataset page; dataset paper, section 4.2

Recording dates and coverage were checked against the dataset page and the dataset paper; the dataset's authors never recorded when or where the source recordings were made, so both entries stay unknown.

1.3 Labels and transformations

- Every excerpt carries a class label in the metadata CSV, as a classID from 0 to 9 and a class name. The labels were produced by the dataset authors: they marked the start and end time of every class occurrence in each source recording in Audacity and rated its salience as perceived foreground or background, labeling 3075 occurrences over 18.5 hours (dataset paper, section 3). Salience is a subjective rating in which 1 means foreground and 2 means background (dataset page).

- On disk, this product changes nothing: the files are used exactly as distributed, and the manifest pins every file by SHA-256 digest.
- In memory, the loader converts every WAV subtype to int16. Subtypes up to 16 bits are read directly, while wider and float subtypes are read at native precision and then rounded and clipped, at most 1/2 LSB of error per sample (p2 loader documentation). Nothing is resampled, so sample rate, bit depth and channel count stay as uploaded (dataset page).
- The only fields the product derives are its own compression metrics (sizes, ratio, RMSE, timing, memory), written to output/results.json; no dataset fields are derived or stored.
- The original data is kept, because the pipeline writes only into the output directory.

1.4 Rights and privacy

Question	Answer	Source
Licence	CC BY-NC 4.0 (non-commercial), as recorded from the Kaggle mirror during the P1 audit; the dataset site does not restate licence terms on its pages	P1 report; dataset page
Attribution	cite the dataset paper (see sources above); the dataset page asks that the dataset be acknowledged in academic research	publications page
Personal or sensitive data	unknown; the excerpts are field recordings of urban environments and the metadata CSV stores no personal fields	dataset page; dataset paper
Access	public download from the dataset site (web form or the soundata package) and from the Kaggle mirror	dataset page; P1 report
Retention and deletion	the working copy is kept for the duration of the course and deleted after grading	our policy
Retrieval of this copy	retrieved from the Kaggle mirror on 25 August 2026	P1 report

Whether identifiable persons are audible was checked against the dataset page and the dataset paper; the dataset's authors never published anything about consent or audible speech, so the entry stays unknown.

1.5 Quality and integrity

- Units and valid ranges: excerpt duration at most 4 s, with sample rate, bit depth and channels varying per file as uploaded; salience 1 or 2; folds 1-10; classID 0-9 (dataset page). After loading, the audio contract of section 2 applies: int16, shape (frames, channels), positive sample rate.
- The measured values are recorded by the QA report in section 3: verified digests, unreadable files, duplicate files, sample rates, channel counts, subtypes, durations, and the metadata CSV checks.

- Missing values appear in the metadata CSV as empty cells, which the QA check counts per column when the CSV is present. In audio, an unreadable or invalid file surfaces as a load error.
- Byte-identical files are detected by comparing SHA-256 digests across all checked files, and duplicate metadata rows by comparing full rows.
- Every file of the release has an entry in manifest/urbansound8k_manifest.json with its relative path, byte size and SHA-256 digest. `python run.py --verify-only` recomputes and compares them for every present file, and `python run.py` verifies the input clip before any processing; a digest mismatch aborts the run.

1.6 Use and maintenance

- **Versioning:** This repository is versioned with Git, and the manifest declares version `1.0`. The dataset release does not publish a version number, so its version is **unknown** (checked against the dataset page and dataset paper).
- **Maintainers:** *Katharina Kivle* and *Nikolai O. Stubø*, for the duration of the IDIG4120 course.
- **Decision:** Use with controls: verify file digests before processing, convert audio to `int16` when loading, do not resample, and preserve the predefined folds. Label correctness has not been verified, and the licence restricts commercial use.

In [2]:
```
per_top_level = {}
for entry in manifest["files"]:
    top = entry["path"].split("/")[0]
    per_top_level[top] = per_top_level.get(top, 0) + 1

print("dataset:", manifest["dataset"], "| manifest version:", manifest["manifest_ve
print("total entries:", len(manifest["files"]))
for name in sorted(per_top_level):
    print(f"  {name}: {per_top_level[name]}")
print("total bytes covered by the manifest:", sum(f["size_bytes"] for f in manifest
```

```
dataset: UrbanSound8K | manifest version: 1.0
total entries: 8733
  UrbanSound8K.csv: 1
  fold1: 873
  fold10: 837
  fold2: 888
  fold3: 925
  fold4: 990
  fold5: 936
  fold6: 823
  fold7: 838
  fold8: 806
  fold9: 816
total bytes covered by the manifest: 7089137662
```

The counts above come from the manifest: it covers the CSV plus all 8,732 excerpts, and the per-fold counts match the published fold sizes on the dataset page.

2. Schema

Three layers have a contract: the metadata CSV, the audio after loading, and the APAC container the pipeline produces.

Metadata CSV (UrbanSound8K.csv)

Column	Type	Meaning	Valid range
slice_file_name	string	excerpt file name, format [fsID]-[classID]-[occurrenceID]-[sliceID].wav	matches one WAV file inside a fold
fsID	integer	Freesound ID of the source recording	344-209992
start	number	excerpt start time in the source recording (s)	0.0-600.125356 seconds
end	number	excerpt end time in the source recording (s)	end > start
salience	integer	subjective salience rating	1 (foreground) or 2 (background)
fold	integer	predefined fold	1-10
classID	integer	sound class identifier	0-9
class	string	sound class name	one of the ten classes

Source: UrbanSound8K dataset page and UrbanSound8K.csv. The ranges for fsID and start are the observed minimum and maximum values from the metadata CSV in the full dataset.

Audio contract after loading

Property	Contract
dtype	int16
shape	(frames, channels), 2D; at least one frame, at least one channel
sample rate	positive integer, taken from the file header
conversion	subtypes up to 16 bits read directly; wider and float subtypes read at native precision, then round and clip to int16, at most 1/2 LSB of error per sample
on disk	unchanged: no resampling, no rewriting

Enforced by p3.pipeline.check_shape before any processing; a violation raises InvalidShape. Source: p2 loader and validation code.

APAC container

A 33-byte big-endian header holds the magic "APAC", version 1, the header length, the sample rate, the frame count, the channels, the predictor and its order, the block size, the quantization step, the DCT mode, window size and parameter, and the entropy coder. After it comes the data section, a sequence of blocks, each starting with a control byte that marks the block as raw, constant, or Rice-coded. The full specification is p2/src/audio_codec/codec/format.py. Source: p2 documentation.

---

3. Quality assurance report

This report checks the UrbanSound8K copy configured in `config.json` for file integrity, duplicates, audio readability and technical ranges, and metadata consistency when the metadata CSV is present.

Scope: The recorded run scanned a full copy of the release. The repository itself contains part of fold1 in `p2/data/raw` 253 of 873 excerpts, including the configured clip. Point `dataset_dir` in `config.json` to a complete copy of the release to repeat the checks. The manifest defines the expected files.

Checks performed

1. **File integrity:** Compare the size and SHA-256 digest of every present file against `manifest/urbansound8k_manifest.json`.
2. **Unexpected files:** Identify files on disk that are not listed in the manifest.
3. **Duplicates:** Detect byte-identical files using the digests computed during the integrity check.
4. **Audio properties:** Check each WAV file's readability, sample rate, channel count, duration, and subtype.
5. **Metadata CSV:** Report record counts, missing values, duplicate rows, and valid ranges. This check runs only when `UrbanSound8K.csv` is present; otherwise, it is reported as skipped.

These checks identify technical problems; they do not verify that class labels accurately describe the audio.

3.1 File integrity

In [3]:
```
summary = integrity.verify_dataset(dataset_dir, manifest)

print("manifest entries:", summary["manifest_files"])
print("verified, size and sha256 match:", summary["verified"])
print("digest mismatches:", len(summary["mismatched"]), summary["mismatched"][:5])
```

```
print("absent, not under the dataset dir:", summary["absent"])
print("unexpected files:", len(summary["unexpected"]), summary["unexpected"][:5])
```

```
manifest entries: 8733
verified, size and sha256 match: 8733
digest mismatches: 0 []
absent, not under the dataset dir: 0
unexpected files: 0 []
```

The repository can carry a subset of the release, so files that are absent are reported without being treated as errors. A digest mismatch or an unexpected file is an integrity failure, and `python run.py --verify-only` reports it.

3.2 Duplicates

In [4]:
```
by_digest = {}
for rel, sha in summary["digests"].items():
    by_digest.setdefault(sha, []).append(rel)
duplicates = {sha: rels for sha, rels in by_digest.items() if len(rels) > 1}

print("files compared:", len(summary["digests"]))
print("byte-identical duplicate files:", len(duplicates))
for sha, rels in list(duplicates.items())[:5]:
    print(" ", sha[:16], rels)
```

```
files compared: 8733
byte-identical duplicate files: 0
```

3.3 Audio properties

The dataset page states excerpts of at most 4 s, with sample rate, bit depth and channels as uploaded. The duration range check uses a 4.1 s tolerance, so an excerpt slightly past the stated maximum is not flagged.

In [5]:
```
records, unreadable = [], []
for entry in manifest["files"]:
    rel = entry["path"]
    if not rel.endswith(".wav"):
        continue
    path = dataset_dir / rel
    if not path.is_file():
        continue
    try:
        info = sf.info(path)
    except Exception as exc:
        unreadable.append((rel, str(exc)))
        continue
    records.append({
        "file": rel,
        "sample_rate": info.samplerate,
        "channels": info.channels,
        "frames": info.frames,
        "duration_s": info.duration,
```

```
        "subtype": info.subtype,
    })

audio = pd.DataFrame(records)
print("audio files inspected:", len(audio))
print("unreadable files:", len(unreadable))
```

```
audio files inspected: 8732
unreadable files: 0
```

In [6]:
```
invalid = audio[
    (audio["sample_rate"] <= 0)
    | (audio["channels"] <= 0)
    | (audio["frames"] <= 0)
    | (audio["duration_s"] <= 0)
    | (audio["duration_s"] > 4.1)
]
print("files outside the valid ranges:", len(invalid))
print()
print("sampling rates:")
display(audio["sample_rate"].value_counts().sort_index())
print("channel counts:")
display(audio["channels"].value_counts().sort_index())
print("WAV subtypes:")
display(audio["subtype"].value_counts())
print("duration in seconds:")
display(audio["duration_s"].describe())
```

```
files outside the valid ranges: 0

sampling rates:
sample_rate
8000        12
11024        7
11025       39
16000       45
22050       44
24000       82
32000        4
44100     5370
48000     2502
96000      610
192000      17
Name: count, dtype: int64
channel counts:
channels
1     739
2    7993
Name: count, dtype: int64
WAV subtypes:
```

```
subtype
PCM_16       5758
PCM_24       2753
FLOAT         169
PCM_U8         43
MS_ADPCM        8
IMA_ADPCM       1
Name: count, dtype: int64
duration in seconds:
count    8732.000000
mean        3.607522
std         0.974394
min         0.050000
25%         4.000000
50%         4.000000
75%         4.000000
max         4.036647
Name: duration_s, dtype: float64
```

3.4 Metadata CSV

In [7]:
```python
csv_path = dataset_dir / "UrbanSound8K.csv"
csv_entry = next(f for f in manifest["files"] if f["path"] == "UrbanSound8K.csv")

if csv_path.is_file():
    integrity.verify_file(csv_path, csv_entry)
    meta = pd.read_csv(csv_path)
    print("verified against the manifest:", csv_entry["sha256"])
    print("metadata records:", len(meta))
    print("columns:", list(meta.columns))
    print("missing values per column:")
    display(meta.isna().sum())
    print("duplicate rows:", int(meta.duplicated().sum()))
    print("duplicate file names:", int(meta["slice_file_name"].duplicated().sum()))
    print("folds outside 1-10:", int((~meta["fold"].between(1, 10)).sum()))
    print("classIDs outside 0-9:", int((~meta["classID"].between(0, 9)).sum()))
    print("salience values outside 1-2:", int((~meta["salience"].isin([1, 2])).sum(
    print("rows with end <= start:", int((meta["end"] <= meta["start"]).sum()))
    print()
    print("records per class:")
    display(meta["class"].value_counts())
    print("records per fold:")
    display(meta["fold"].value_counts().sort_index())
else:
    print("skipped: UrbanSound8K.csv is not present under the dataset dir")
    print("the manifest entry for the file is:", csv_entry)
```

```
verified against the manifest: b207f7a53f9e5eb02a5dd71d58fb8b66c88c24c0637b926dc72d9
d400a6a2354
metadata records: 8732
columns: ['slice_file_name', 'fsID', 'start', 'end', 'salience', 'fold', 'classID',
'class']
missing values per column:
```

```
slice_file_name    0
fsID               0
start              0
end                0
salience           0
fold               0
classID            0
class              0
dtype: int64
duplicate rows: 0
duplicate file names: 0
folds outside 1-10: 0
classIDs outside 0-9: 0
salience values outside 1-2: 0
rows with end <= start: 0

records per class:
class
dog_bark            1000
children_playing    1000
air_conditioner     1000
street_music        1000
engine_idling       1000
jackhammer          1000
drilling            1000
siren                929
car_horn             429
gun_shot             374
Name: count, dtype: int64
records per fold:
fold
1     873
2     888
3     925
4     990
5     936
6     823
7     838
8     806
9     816
10    837
Name: count, dtype: int64
```

3.5 Summary

The counts above are the report. They cover verified files, digest mismatches, unexpected files, byte-identical duplicates, unreadable files, files outside the valid ranges, and the metadata checks when the CSV was present.

---

4. Pipeline and results

4.1 What the pipeline does

The entry point is run.py next to this notebook, started with `python run.py`. It runs one clip from input to output:

1. **Preflight:** Confirm the config, manifest, dataset directory, and input clip exist. If the input is missing, abort before processing.
2. **Verify:** Check the clip's size and SHA-256 digest against the manifest. Abort if they do not match.
3. **Load:** Check that the clip meets the section 2 audio contract: `int16` samples with shape `(frames, channels)`, consistent with the file header. Abort if it does not.
4. **Encode:** Encode with APAC using the `quant_step` from `config.json` (`1` means lossless).
5. **Decode:** Decode the APAC output.
6. **Check:** For a lossless run, confirm the decoded samples exactly match the loaded samples. Fail the run otherwise.
7. **Save:** Write `compressed.apac`, `decoded.wav`, and `results.json`. Each file is completed under a temporary name and then renamed into place, so an interrupted run does not leave partial files.

Every step appends one line to output/run.log. The pipeline has no random steps: the same input, config and codec version produce the same output bytes, so no seed is needed.

The cell below runs the entry point with the configuration from config.json.

In [8]:
```
import subprocess
import sys
from pathlib import Path

root = Path.cwd()
while not (root / "run.py").is_file():
    root = root.parent

run = subprocess.run([sys.executable, "run.py"], capture_output=True, text=True, cw
print(run.stdout)
print("exit code:", run.returncode)
```

```
run started: input fold1/124489-9-0-12.wav, quant_step 1
verify: sha256 ok for fold1/124489-9-0-12.wav
load: fold1/124489-9-0-12.wav frames=176400 channels=2 rate=44100 subtype=PCM_16
encode: 476353 bytes in 0.099s
decode: 176400 frames in 0.693s
save: compressed.apac, decoded.wav, results.json -> output
run finished: bit_exact=True ratio=1.481 in 0.81s
input_file: fold1/124489-9-0-12.wav
input_sha256: adab612f876afd5f13d2ef8d39d64a466507c77d6d00aeee3f880542b6386946
sample_rate: 44100
frames: 176400
channels: 2
subtype: PCM_16
quant_step: 1
wav_bytes: 705644
apac_bytes: 476353
compression_ratio: 1.481
space_saving_percent: 32.49
bit_exact: True
rmse: 0.0
encode_seconds: 0.0989
decode_seconds: 0.693
encode_peak_bytes: 2823528
decode_peak_bytes: 16842970
run_seconds: 0.807
timestamp_utc: 2026-11-06T18:23:42+00:00
environment: {'python': '3.13.7', 'numpy': '2.3.3', 'soundfile': '0.13.1'}

exit code: 0
```

4.2 Results

The same metrics are saved as output/results.json by the run above.

In [9]:
```python
import json

results = json.loads((root / "output" / "results.json").read_text(encoding="utf-8")
for key, value in results.items():
    print(f"{key}: {value}")
```

```
input_file: fold1/124489-9-0-12.wav
input_sha256: adab612f876afd5f13d2ef8d39d64a466507c77d6d00aeee3f880542b6386946
sample_rate: 44100
frames: 176400
channels: 2
subtype: PCM_16
quant_step: 1
wav_bytes: 705644
apac_bytes: 476353
compression_ratio: 1.481
space_saving_percent: 32.49
bit_exact: True
rmse: 0.0
encode_seconds: 0.0989
decode_seconds: 0.693
encode_peak_bytes: 2823528
decode_peak_bytes: 16842970
run_seconds: 0.807
timestamp_utc: 2026-11-06T18:23:42+00:00
environment: {'python': '3.13.7', 'numpy': '2.3.3', 'soundfile': '0.13.1'}
```

4.3 Timing and memory evidence

Each row compares a storage option for the same clip: PCM WAV, lossless APAC (quantization step 1), or lossy APAC (step 4, the FLAC crossover identified in the P2 evaluation). Wall-clock time is measured with `time.perf_counter`, and peak memory with `tracemalloc` during each stage.

These numbers stem from a single run on a single machine.

In [10]:
```
config = cfg.load_config(root)
bench = pipeline.benchmark(root, config)
print("input:", bench["input_file"])
display(pd.DataFrame(bench["rows"]))
```

```
input: fold1/124489-9-0-12.wav
```

	storage	bytes	encode_s	decode_s	encode_peak_mb	decode_peak_mb	bit_exact	
0	WAV PCM_16 (original file)	705644	NaN	NaN	NaN	NaN	True	0.(
1	APAC quant_step 1	476353	0.0987	0.7502	2.69	16.06	True	0.(
2	APAC quant_step 4	387775	2.5589	0.7197	2.69	15.97	False	0.8

4.4 Failure handling

The four required failure cases are covered by tests, one fixture each:

Case	What happens	Where
Missing input	preflight names the missing path, nothing runs	*test_missing_input_aborts_before_anything_runs*
Digest mismatch	verification aborts before encoding, no outputs	*test_digest_mismatch_aborts_before_encoding*
Invalid shape	the contract rejects 1D, non-int16 and header-mismatched arrays	*test_invalid_shape_is_rejected*
Interrupted run	temp files are removed, nothing partial is kept, the log records the step	*test_interrupted_run_leaves_no_partial_outputs*

The cell below runs the test suite; the transcript shows that all cases pass.

In [11]:
```
tests = subprocess.run([sys.executable, "-m", "pytest", "tests", "-q"],
                       capture_output=True, text=True, cwd=root)
print(tests.stdout)
print("exit code:", tests.returncode)
```

```
.........                                                                [100%]
9 passed in 1.04s

exit code: 0
```

---

5. Known limitations

- Coverage: the repository carries part of fold1 inside p2/data/raw (253 of the 873 excerpts, including the configured clip), and the default pipeline run uses one clip from it. The manifest covers the full release, and `python run.py --verify-only --dataset-dir PATH` verifies any complete copy against it.
- Timing and memory numbers vary between machines and runs.
- No check verifies that the class labels describe the audio correctly, and the pipeline does not use the labels.
- The pipeline converts wider-than-16-bit and float audio to int16, so `decoded.wav` may differ from the original.
- The licence (CC BY-NC 4.0, per the P1 audit) restricts commercial use of the data.

---

6. Who did what

The project is done in collaboration and with communication throughout the weeks it was assigned. Through the weeks both of us had work in different shifts. For P3, Katharina started, getting half of the assignment ready for review. Later Nikolai took over, looking over what Katharina had made. He then continued where she left off. Later Katharina would go through the last implementations, look over and finish. We both looked over at the end before the final submission.